Community Pharmacy Inventory and Sales Management System

Report 2: UML Class Diagram Report

Kigali Independent University (ULK) | Computer Science

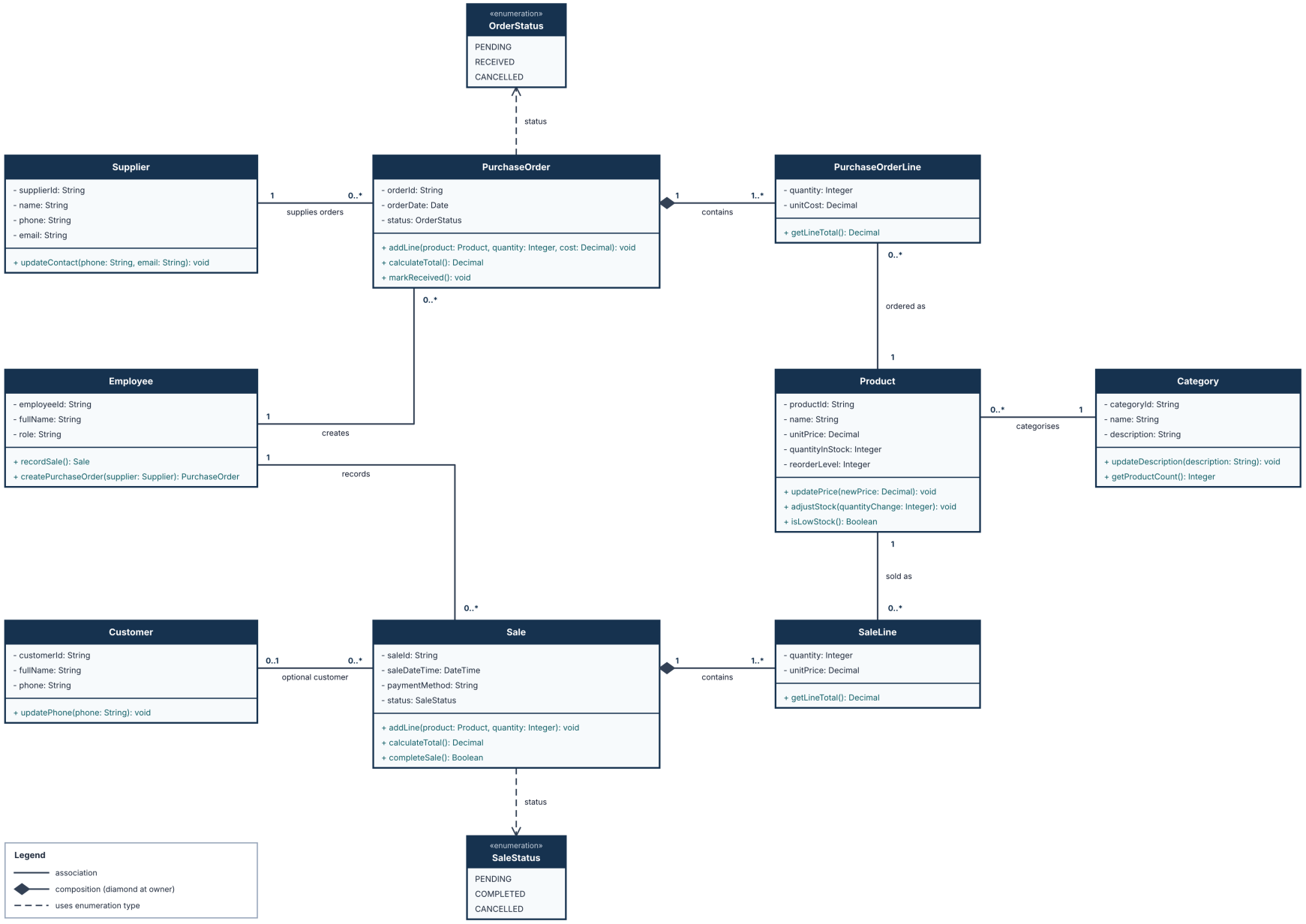
Anas Yaseir (202512831) and Asim Mohamed (202511198)

1. Diagram

The diagram on the next page models the nine business classes, two enumeration types and their relationships. Multiplicities are written at each end of every relationship, and composition (filled diamond at the owner end) is used only for transaction line items whose lifecycle belongs to the owning transaction. No relationship line crosses a class box or another line. The diagram is also supplied as a vector PDF and an editable draw.io file for zooming into class details.

Community Pharmacy Inventory and Sales Management System

UML class diagram | Kigali Independent University (ULK) | Anas Yaseir (202512831) and Asim Mohamed (202511198)



2. Class Descriptions, Attributes and Operations

Product

Responsibility: Represents a medicine or other stocked pharmacy item.

Attributes: - productId: String; - name: String; - unitPrice: Decimal; - quantityInStock: Integer; - reorderLevel: Integer

Operations: + updatePrice(newPrice: Decimal): void; + adjustStock(quantityChange: Integer): void; + isLowStock(): Boolean

Category

Responsibility: Groups products by a pharmacy category.

Attributes: - categoryId: String; - name: String; - description: String

Operations: + updateDescription(description: String): void; + getProductCount(): Integer

Supplier

Responsibility: Stores details of a business that supplies pharmacy products.

Attributes: - supplierId: String; - name: String; - phone: String; - email: String

Operations: + updateContact(phone: String, email: String): void

PurchaseOrder

Responsibility: Records an order placed with a supplier.

Attributes: - orderId: String; - orderDate: Date; - status: OrderStatus

Operations: + addLine(product: Product, quantity: Integer, cost: Decimal): void; + calculateTotal(): Decimal; + markReceived(): void

PurchaseOrderLine

Responsibility: Records one product and quantity within a purchase order.

Attributes: - quantity: Integer; - unitCost: Decimal

Operations: + getLineTotal(): Decimal

Sale

Responsibility: Records a customer transaction processed by an employee.

Attributes: - saleId: String; - saleDateTime: DateTime; - paymentMethod: String; - status: SaleStatus

Operations: + addLine(product: Product, quantity: Integer): void; + calculateTotal(): Decimal; + completeSale(): Boolean

SaleLine

Responsibility: Records one product, quantity and sale price within a sale.

Attributes: - quantity: Integer; - unitPrice: Decimal

Operations: + getLineTotal(): Decimal

Customer

Responsibility: Stores optional customer details for a sale.

Attributes: - customerId: String; - fullName: String; - phone: String

Operations: + updatePhone(phone: String): void

Employee

Responsibility: Represents a staff member who records and manages transactions.

Attributes: - employeeId: String; - fullName: String; - role: String

Operations: + recordSale(): Sale; + createPurchaseOrder(supplier: Supplier): PurchaseOrder

Enumerations

OrderStatus («enumeration»): Lifecycle state of a purchase order. Values: PENDING, RECEIVED, CANCELLED. It is used as the type of the *status* attribute and is shown with a dashed dependency arrow.

SaleStatus («enumeration»): Lifecycle state of a sale. Values: PENDING, COMPLETED, CANCELLED. It is used as the type of the *status* attribute and is shown with a dashed dependency arrow.

3. Relationship Analysis

Category — Product (“categorises”): Association. One Category categorises zero or many Products (1 to 0..*); each Product belongs to exactly one Category. A category may temporarily have no products.

Supplier — PurchaseOrder (“supplies orders”): Association. One Supplier may receive zero or many PurchaseOrders (1 to 0..*); each PurchaseOrder is placed with exactly one Supplier.

PurchaseOrder — PurchaseOrderLine (“contains”): Composition. One PurchaseOrder contains one or more PurchaseOrderLines (1 to 1..*); each line belongs to exactly one order. The filled diamond is at PurchaseOrder.

Product — PurchaseOrderLine (“ordered as”): Association. One Product may appear on zero or many PurchaseOrderLines (1 to 0..*); each PurchaseOrderLine refers to exactly one Product.

Employee — PurchaseOrder (“creates”): Association. One Employee may create zero or many PurchaseOrders (1 to 0..*); each PurchaseOrder is created by exactly one Employee.

Employee — Sale (“records”): Association. One Employee may record zero or many Sales (1 to 0..*); each Sale is recorded by exactly one Employee.

Customer — Sale (“optional customer”): Association. A Customer may be associated with zero or many Sales (0..* at the Sale end); a Sale may have zero or one Customer (0..1 at the Customer end). Customer identification is optional.

Sale — SaleLine (“contains”): Composition. One Sale contains one or more SaleLines (1 to 1..*); each line belongs to exactly one Sale. The filled diamond is at Sale.

Product — SaleLine (“sold as”): Association. One Product may appear on zero or many SaleLines (1 to 0..*); each SaleLine refers to exactly one Product.

PurchaseOrder → **OrderStatus and Sale** → **SaleStatus**: Dependency (dashed arrow). The status attribute takes its value from the enumeration; this is a type usage, not a business association, so it carries no multiplicity.

4. UML Notation

A class rectangle is divided into a name, attribute, and operation compartment. The minus sign (-) means private visibility; the plus sign (+) means public visibility. Multiplicity 1 means exactly one; 0..1 means optional, at most one; 0..* means zero or many; and 1..* means one or many. A filled diamond marks composition and is placed at the whole/owner end. Ordinary solid association lines show domain links without asserting ownership. A box marked «enumeration» lists the fixed set of values a type may take.

5. Modelling Decisions and Constraints

PurchaseOrderLine and SaleLine are ordinary classes rather than bare many-to-many links because each relationship has its own attributes (quantity and unit cost or sale price). Product stores the current unit price and stock quantity, while SaleLine stores the price charged at the time of a sale so historical totals are not forced to change when the product price changes. PurchaseOrderLine stores the cost for that order for the same reason. The model does not include inheritance because the current requirements do not justify specialised subclasses. Aggregation is omitted because it would not communicate a stronger or clearer rule than the ordinary associations used here.

Order and sale statuses are modelled as enumerations (OrderStatus, SaleStatus) so that the allowed values are defined rather than free text. Creating a purchase order is the responsibility of Employee only, and it takes the Supplier as a parameter (createPurchaseOrder(supplier: Supplier)); the Supplier class therefore keeps only its own contact-update operation. Category keeps simple operations about its own data (updateDescription, getProductCount) because the Category–Product association already represents which products belong to it.

6. Multiplicity and Integrity Notes

The single category per product is a deliberate simplifying assumption. A future system could permit multiple categories, but that would require a different multiplicity. Transaction totals should be calculated from line totals. Quantities should be positive and prices/costs should be non-negative; these are business constraints to validate in an implementation. The model treats each line as referring to one Product and does not represent batches, expiry dates, returns, discounts, or taxes.

7. Conclusion

The class diagram represents the core records for a small pharmacy's stock, purchasing, and sales workflows. Its transaction-line classes preserve line-level data, its multiplicities express participation rules, composition identifies line items as parts of their transactions, and enumerations define the allowed statuses. The model is intentionally limited to the stated scope and can be expanded if additional requirements are introduced.